Student Learning Management System

Functional & Non-Functional Requirements

Upgrade2Architect Phase 3 — Version 1.0

Document title	Student LMS — FR / NFR Requirements Document
Version	1.0
Status	Draft
Author	Pratyush Chandra Jha — 2154615
Date	22-May-2026
Classification	Confidential

1. Functional Requirements

The following requirements (F001–F011) define all platform features spanning UI, API integration, authentication, user management, course management, assessments, progress tracking, grading, notifications, event logging, and code quality.

F001 — Responsive UI

F001	UI Component	Requirements & Implementation Notes
F001	**Student Dashboard**	Display enrolled courses, upcoming deadlines, recent grades, and progress percentage per course. React component polls Progress and Assessment services at a configurable interval (default 60s, min 30s). Progress bars rendered client-side from completion % returned in API response.
F001	**Course Player**	Video player with chapter navigation, timestamped notes, and bookmarks. Video streamed from S3 via CloudFront pre-signed URLs (satisfies 30s NFR001 SLA). Notes and bookmarks persisted via POST /progress/notes and POST /progress/bookmarks; loaded on player init.
F001	**Assessment Interface**	Timed quiz interface with visible countdown; auto-submit fires on expiry via client-side setInterval synced to server-issued start timestamp. EventBridge backup Lambda triggers auto-submit if client fails. MaxAttempts enforced server-side by Assessment service before allowing a new attempt.
F001	**Search & Filter**	Search courses by title, instructor, or category. Filter by difficulty level and estimated duration. GET /courses?search=&category;=&difficulty;=&maxDuration;= served by Course service. Results cached in ElastiCache Redis (10-min TTL). Frontend debounces input 300ms before firing request.
F001	**Auto Refresh**	Progress and grade data refreshed at a configurable interval (default 60s, min 30s) via React useEffect + setInterval. WebSocket push from API Gateway triggers an immediate UI update on grade release, bypassing the polling cycle for time-sensitive events.

F002 — API Integration

F002	API	Requirements & Implementation Notes
F002	**Video Storage API**	Integrate with Amazon S3 for course video uploading and streaming. Instructors upload via Course service which generates a pre-signed S3 PUT URL. Students stream via CloudFront pre-signed GET URLs served directly from S3, bypassing the API tier. Satisfies 30s content load SLA (NFR001).
F002	**Email Notification API**	Amazon SES triggered by Notification service for: enrolment confirmation, assignment deadline reminders (48hr advance), and grade release. Events consumed from SQS queue published by Enrolment, Assessment, and Progress services via SNS fanout.
F002	**Response Handling**	All REST API responses return JSON. Course content endpoints return structured lesson metadata. Quiz result endpoints return per-question scores, correct answers, and total grade. Progress endpoints return completion percentage, lesson view history, and certificate URL where applicable. Standardised error response schema applied globally via @ControllerAdvice.
F002	**Caching**	Course metadata (course:{id}:metadata) and module list (course:{id}:modules) cached in ElastiCache Redis with 10-minute TTL. Cache invalidated on any course update or publish event. Reduces RDS read load and improves API P95 response time.

F003 — User Authentication and Authorisation

F003	Feature	Requirements & Implementation Notes
F003	**Secure Login & Signup**	Registration and login supported for all three user types: Admin, Instructor, and Student. Auth service exposes POST /auth/register and POST /auth/login. API Gateway validates JWT on every subsequent request before forwarding to downstream services.
F003	**Password Security**	Passwords hashed using BCrypt with salt rounds >= 12 before persisting to RDS PostgreSQL. Plain-text passwords are never stored or logged. Password reset flow uses time-limited signed tokens.
F003	**Session Management**	JWT-based authentication: access token TTL 15 minutes, refresh token TTL 7 days. Refresh tokens stored in ElastiCache Redis; invalidated on logout (token blacklist). Refresh token rotation enforced on every use.
F003	**Role-Based Access Control**	Three roles: Admin (full platform access), Instructor (own courses, grading, announcements), Student (enrolled content, own submissions, own profile). Role claim embedded in JWT; each microservice enforces RBAC independently before processing requests.

F004 — User Management

F004	Role	Requirements & Implementation Notes
F004	**Admin**	Create, update, and deactivate user accounts. Assign and change user roles. View all user profiles with search and filter. User service exposes CRUD endpoints under /api/users accessible only to Admin role JWT.
F004	**Instructor**	Manage own courses (create, update, publish, archive). View list of students enrolled in own courses. Grade assignment submissions and provide written feedback. Access restricted to own course data; cross-instructor data access rejected at service layer.
F004	**Student**	Manage own profile (name, avatar, notification preferences). View enrolment history and status of all enrolled courses. Track grades per assessment and cumulative course grade. No access to other students' data; user ID validated against JWT subject claim on every request.

F005 — Course Management

F005	Feature	Requirements & Implementation Notes
F005	**Course Lifecycle**	Instructors can create, update, publish, and archive courses. Approval workflow: Draft -> Pending -> Published -> Archived. Admin must approve before a course appears in the public catalogue. Course service manages all state transitions with event published to SNS on each status change.
F005	**Course Structure**	Courses consist of Modules; each Module contains one or more Lessons. Lesson content types supported: text, video (S3/CloudFront), and PDF (S3). Relational schema: Courses(1)->Modules(many)->Lessons(many). Course service owns CRUD for all three levels.
F005	**Course Metadata**	Each course includes: category, difficulty level (Beginner, Intermediate, Advanced), and estimated duration. Metadata cached in ElastiCache Redis (10-min TTL). Searchable and filterable via GET /courses query parameters.
F005	**Enrolment**	Students can enrol in free courses directly. Paid course enrolment triggers payment verification via Payment Gateway integration in Enrolment service. On successful enrolment, an event is published to SNS which triggers an email confirmation via SES.

F006 — Assessments and Quizzes

F006	Feature	Requirements & Implementation Notes
F006	**Question Types**	Instructors create quizzes with three question types: multiple choice, true/false, and short answer. Assessment service stores question bank per course. Question type determines grading path: objective types auto-graded; short answer queued for instructor review.
F006	**Timed Quizzes**	Each quiz has a configurable duration per attempt. Client-side countdown synced to server-issued start timestamp. Auto-submit triggered on expiry by React client; EventBridge Lambda provides backup trigger if client fails to submit within the deadline window.
F006	**Grading**	MCQ and true/false questions auto-graded by Assessment service Lambda on submission. Short answer submissions flagged as pending and appear in Instructor grading queue. Instructor grades via PUT /api/submissions/{id}/grade with score and written feedback.
F006	**Retake Attempts**	Configurable MaxAttempts per quiz set by Instructor at quiz creation. Assessment service validates attempt count before allowing a new submission. Highest or latest score policy configurable per quiz.
F006	**Assignment Submissions**	File upload support for PDF, DOCX, and image formats. Student uploads via pre-signed S3 PUT URL issued by Assessment service. File metadata (S3 key, filename, upload timestamp) stored in RDS Submissions table. Instructor downloads via pre-signed S3 GET URL.

F007 — Progress Tracking

F007	Feature	Requirements & Implementation Notes
F007	**Completion Tracking**	Progress service tracks completion percentage per course and per module. Completion % = (lessons viewed + assessments submitted) / total items. Stored in RDS Enrollments table. Lesson view events streamed via Kinesis -> Lambda -> DynamoDB for high write throughput.
F007	**Activity Recording**	Every lesson view, quiz attempt, and assignment submission fires an event to Kinesis Data Streams. Lambda consumer persists events to DynamoDB (Logs table) and updates Progress in RDS. Provides full audit trail and input data for cohort reports.
F007	**Certificate Generation**	When course completion reaches 100%, Progress service publishes a CertificateRequired event to SNS. Lambda subscriber generates a PDF certificate (student name, course title, completion date), stores it in S3, and updates the Enrollment record with the certificate URL. Student receives in-app and email notification with download link. Target SLA: < 30 seconds.
F007	**Cohort Reports**	Instructors can view progress reports for all students enrolled in their courses. Admins can view cross-course cohort reports. Aggregate queries run against RDS read replica; summary data cached in ElastiCache for dashboard performance.

F008 — Grading and Feedback

F008	Feature	Requirements & Implementation Notes
F008	**Grade Display**	Students view per-assessment scores and a cumulative course grade on their dashboard. Cumulative grade = weighted average of all graded assessments in the course. Assessment service exposes GET /api/submissions and GET /api/progress endpoints.
F008	**Written Feedback**	Instructors provide written feedback when grading short-answer questions and assignment submissions. Feedback stored in RDS Submissions table (feedback column). Visible to the student via their submission detail view once grade is released.
F008	**Grade Release Notifications**	On grade release, Assessment service publishes a GradeReleased event to SNS. Notification service fans out to: SES (email to student) and API Gateway WebSocket (in-app push). Student sees grade notification in real-time without waiting for next polling cycle.

F008	Feature	Requirements & Implementation Notes
F008	**Admin Grade Reports**	Admins access grade distribution reports per course. Reports show score buckets (0-49, 50-69, 70-84, 85-100), mean, median, and pass rate. Queries run against RDS read replica; results cached for 10 minutes.

F009 — Notifications and Announcements

F009	Feature	Requirements & Implementation Notes
F009	**Notification Triggers**	In-app (WebSocket) and email (SES) notifications fired for: enrolment confirmation, new course content published, upcoming assignment deadline (48-hour advance warning via EventBridge Scheduler), and grade release. All triggers published as SNS events and consumed by Notification service.
F009	**Course Announcements**	Instructors post announcements via POST /api/courses/{id}/announcements. Announcement creation publishes an event to SNS; Notification service fans out to all enrolled students via WebSocket (in-app) and SES (email) based on their per-course preferences.
F009	**Student Preferences**	Students configure notification preferences per course (email on/off, in-app on/off). Preferences stored in ElastiCache Redis (TTL 60 minutes) and persisted to RDS. Notification service checks preferences before dispatching each notification type.

F010 — Event Logging

F010	Feature	Requirements & Implementation Notes
F010	**Logged Events**	All platform events logged: login, enrolment, lesson views, quiz attempts, assignment submissions, and application errors. Every microservice publishes log events to Kinesis Data Streams; Lambda consumer persists to DynamoDB (Logs table) and indexes in OpenSearch for querying.
F010	**Admin Log Viewer**	Admins filter logs by event type, actor (user ID or name), and date range via GET /api/logs. OpenSearch powers full-text and range queries for fast filtering across large log volumes. Results paginated (default page size 50).
F010	**Log Structure**	Each log entry must include: actorId (user who triggered the event), eventType (LOGIN, ENROL, LESSON_VIEW, QUIZ_ATTEMPT, SUBMISSION, ERROR), description (human-readable summary), and timestamp (ISO 8601 UTC). DynamoDB TTL configured for automatic log expiry per data retention policy.

F011 — Code Quality and Coverage

F011	Feature	Requirements & Implementation Notes
F011	**Static Analysis**	SonarQube enforced on all Java/Spring Boot microservices: 0 blocker issues, 0 critical issues, coverage >= 80%. ESLint enforced on all React/TypeScript frontend code. Both gates run in CodeBuild pipeline; build fails if gate is not met (warning-only for first 2 sprints).
F011	**Coverage Threshold**	API code coverage must exceed 80% as measured by JaCoCo integrated with SonarQube. Coverage gate enforced in CI pipeline; promotion to staging blocked if threshold not met.
F011	**Mandatory Unit Tests**	Unit tests mandatory for: (1) Quiz grading logic — all question types and edge cases. (2) Progress calculation — completion % formula, 100% trigger for certificate generation. (3) Certificate generation trigger — event published correctly on completion, idempotency check. Implemented with JUnit 5 + Mockito; Amazon Q Developer used to accelerate test generation.

2. Non-Functional Requirements

NFR001 defines the non-functional constraints governing error handling, rate limiting, performance SLAs, containerisation, and CI/CD for the LMS platform.

Req ID	Requirement	Design Decision & Implementation
NFR001	**Error / Exception Handling**	Comprehensive error handling throughout the application. Each Spring Boot microservice implements a global @ControllerAdvice exception handler returning a standardised JSON error schema (timestamp, status, errorCode, message, traceId). CloudWatch Alarms trigger on error rate thresholds; X-Ray traces capture all exceptions for root-cause analysis.
NFR001	**Rate Limiting**	API abuse prevention at two layers: (1) API Gateway throttling with configurable burst and steady-state limits per endpoint. (2) AWS WAF rate-based rules per source IP. Clients exceeding limits receive HTTP 429 with a Retry-After header. Limits: 100 req/min unauthenticated, 300 req/min authenticated.
NFR001	**Content Load SLA**	Course content (video, PDF, text) must load within 30 seconds. CloudFront CDN serves static assets from edge locations; S3 pre-signed URLs enable direct browser-to-CDN video streaming, bypassing the application tier entirely. ElastiCache Redis metadata cache reduces API round-trips. CloudFront P95 < 2s for assets.
NFR001	**Containerised Deployment**	All eight Spring Boot microservices are containerised using Docker. Images built and tagged with Git SHA in AWS CodeBuild, pushed to Amazon ECR, and deployed to ECS Fargate with health checks. No EC2 instance management required; Fargate provides serverless container execution.
NFR001	**Modular Codebase & CI/CD**	Microservices architecture enforces modularity — each service has an independent CodePipeline. Pipeline stages: source -> lint -> unit tests -> integration tests -> build -> image push -> quality gate -> staging deploy -> smoke tests -> production deploy (blue/green with automatic rollback). SonarQube quality gate and 80% coverage threshold enforced before any promotion.

3. API Endpoints — Requirements Traceability

The table below maps every REST API endpoint to its corresponding functional requirement. Base URL: https://api.lms.ctsu2a.com/api/v1 | Auth: JWT Bearer token

Authentication — F003

Req	Method	Endpoint	Auth	Description
F003	POST	/auth/register	Public	Register new user account (Admin/Instructor/Student)
F003	POST	/auth/login	Public	Login; receive JWT access + refresh tokens
F003	POST	/auth/logout	JWT	Logout; invalidate refresh token in Redis

User Management — F004

Req	Method	Endpoint	Auth	Description
F004	GET	/users	Admin	List all users with search and filter
F004	GET	/users/{id}	JWT	Get user profile by ID
F004	PUT	/users/{id}	JWT	Update user profile (own or Admin)
F004	DELETE	/users/{id}	Admin	Deactivate user account (soft delete)

Course & Enrolment Management — F005

Req	Method	Endpoint	Auth	Description
F005	GET	/courses	Public	List published courses; supports search, category, difficulty, duration filters
F005	GET	/courses/{id}	Public	Get course details by ID
F005	POST	/courses	Instructor	Create new course (status = DRAFT)
F005	PUT	/courses/{id}	Instructor	Update course details
F005	DELETE	/courses/{id}	Instr/Admin	Archive course (status = ARCHIVED)
F005	PUT	/courses/{id}/publish	Instr/Admin	Submit for approval or approve course
F005	GET	/courses/{id}/modules	JWT	List all modules for a course
F005	POST	/modules	Instructor	Create module within a course
F005	PUT	/modules/{id}	Instructor	Update module details
F005	DELETE	/modules/{id}	Instructor	Delete module (cascades to lessons)
F005	GET	/modules/{id}/lessons	JWT	List lessons within a module
F005	POST	/lessons	Instructor	Create lesson (text / video / PDF)
F005	PUT	/lessons/{id}	Instructor	Update lesson content or metadata

Req	Method	Endpoint	Auth	Description
F005	DELETE	/lessons/{id}	Instructor	Delete lesson
F005	POST	/enrollments	Student	Enrol in a course (free or paid)
F005	GET	/enrollments?student_id={id}	JWT	Get all courses a student is enrolled in
F005	GET	/enrollments?course_id={id}	Instr/Admin	Get all students enrolled in a course
F005	DELETE	/enrollments/{id}	Stud/Admin	Unenrol from a course

Assessments & Submissions — F006 / F008

Req	Method	Endpoint	Auth	Description
F006	GET	/assessments?course_id={id}	JWT	List all assessments for a course
F006	POST	/assessments	Instructor	Create quiz or assignment
F006	PUT	/assessments/{id}	Instructor	Update assessment configuration
F006	DELETE	/assessments/{id}	Instructor	Delete assessment
F006	POST	/submissions	Student	Submit quiz answers or assignment file URL
F006	GET	/submissions?assessment_id={id}	Instr/Admin	Get all submissions for an assessment
F006	GET	/submissions/{id}	JWT	Get a specific submission by ID
F008	PUT	/submissions/{id}/grade	Instructor	Grade submission; add score and written feedback
F006	POST	/files/upload-url	JWT	Request pre-signed S3 PUT URL for assignment file upload

Progress Tracking — F007

Req	Method	Endpoint	Auth	Description
F007	GET	/progress?student_id={id}&course;_id={id}	JWT	Get detailed progress for a student in a course
F007	PUT	/progress/lesson-complete	Student	Mark a lesson as viewed / complete

Notifications & Announcements — F009

Req	Method	Endpoint	Auth	Description
F009	POST	/courses/{id}/announcements	Instructor	Post announcement to all enrolled students
F009	PATCH	/students/{id}/preferences	Student	Update per-course notification preferences
F009	WS	wss://ws.lms…/ws?token={jwt}	JWT	Establish WebSocket connection for real-time in-app notifications

Event Logging — F010

Req	Method	Endpoint	Auth	Description
F010	GET	/logs	Admin	Get filtered log list (event type, actor, date range)
F010	GET	/logs/{id}	Admin	Get a specific log entry by ID
F010	POST	/logs	System	Internal: write a new log entry

File Uploads — F006 (S3)

Req	Method	Endpoint	Auth	Description
F006	POST	/files/upload-url	JWT	Request pre-signed S3 PUT URL for assignment file upload

3.1 Requirements Traceability Summary

Each functional requirement and its corresponding endpoint count at a glance.

Req ID	Requirement Name	API Endpoints	Implementation Layer
F001	Responsive UI	N/A (UI/Config)	React/Next.js Frontend (SPA)
F002	API Integration	N/A (UI/Config)	Course Svc, Notification Svc, ElastiCache Redis
F003	User Authentication & Authorisation	3	Auth Service, API Gateway, Redis
F004	User Management	4	User Service (RDS PostgreSQL)
F005	Course Management	18	Course Svc, Enrolment Svc (RDS, S3, SNS)
F006	Assessments and Quizzes	8	Assessment Svc, Lambda (SQS), S3
F007	Progress Tracking	2	Progress Svc, Lambda (Kinesis, DynamoDB, S3)
F008	Grading and Feedback	1	Assessment Svc, Notification Svc (SNS, SES, WS)
F009	Notifications and Announcements	3	Notification Svc (SNS, SES, WebSocket, EventBridge)
F010	Event Logging	3	Logging Svc (Kinesis, DynamoDB, OpenSearch)
F011	Code Quality and Coverage	N/A (UI/Config)	CodePipeline, CodeBuild, SonarQube, JaCoCo
NFR001	Non-Functional Requirements	N/A (UI/Config)	API Gateway, WAF, CloudFront, ECS Fargate, Docker

Base URL: https://api.lms.ctsu2a.com/api/v1 | Auth: JWT Bearer | WebSocket: wss://ws.lms.ctsu2a.com/ws | All responses: application/json | Errors: RFC 7807 Problem Detail